

Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Carthage
Institut supérieur des technologies de l'information et de la
communication

Chapitre 5: Les fonctions

Plan

1. Introduction
2. La notion de blocs et la portée des identificateurs
3. Les variables globales
4. Les paramètres d'une fonction
5. Exercices



1. Introduction



Il est souhaitable, pour diverses raisons, de décomposer un problème en plusieurs sous-tâches, et de programmer ces sous-tâches comme des blocs indépendants. C'est le concept de la programmation modulaire qui utilise des sous-programmes. En langage C, il y a une seule sorte de sous-programmes : les fonctions.

Voici quelques avantages de la programmation modulaire:

- * Meilleure lisibilité
- * Diminution du risque d'erreurs
- * Réutilisation de modules déjà existants

Structures d'un programme C qui inclut des sous programmes :

-En général, une fonction est définie avant la fonction qui l'appelle comme il est illustré avec la structure suivante.

```
/* inclusion des fichiers (les include)*/  
/*déclaration des variables globales*/  
/*Définition des fonctions*/  
Void main() {  
  { <déclarations des variables locales au main>  
    < instructions>  
  }  
}
```

Lorsqu'une fonction doit être utilisée avant d'être définie, sa déclaration doit précéder l'appel comme il est illustré avec la structure suivante :

```
/* inclusion des fichiers (les include)*/  
/*déclaration des variables globales et des fonctions*/  
Void main() {  
  { <déclarations des variables locales au main>  
    < instructions>  
  }  
  /*Définition des fonctions*/  
}
```

Remarque :

-les instructions peuvent être des appels à des fonctions ou autre traitements.

Exemple 1 : écrire un programme qui permet de calculer et afficher la factorielle d'un entier N (utiliser les sous-programmes).

```
#include <stdio.h>
int reponse;
int factorielle (int N);
void main()
{   int N;
    long f;

    printf("donner valeur de N");
    scanf("%d", &N);
    do
    {
        printf("donner réponse");
        scanf("%d", &reponse);
    } while ((reponse!=1 )&& (reponse!=0));
    if (reponse==1)
    {
        f=factorielle (N);
        printf("factorielle de %d est %ld", N, f);
    }
}

int factorielle (int N)
{ int fact=1;

    for(int i=1; i<=N;i++ )
        fact= fact*i;

    return fact;
}
```

2. La notion de blocs et la portée des identificateurs

Une fonction est un ensemble d'instructions structurées portant un nom et permettant d'effectuer une action bien déterminée. Elle peut être appelée par d'autres fonctions.

Syntaxe :

```
<type> <identificateur > (type1 argument1,type2
argument2,...,typen argumentn)
{
<déclarations locales>

<instructions>

<return>

}
```

<type> : indique le type du résultat fournie par la fonction. Il doit être compatible avec le type de la variable indiquée dans le mot clé `return` et l'appel de la fonction.

- Si une fonction ne fournit pas de résultat, il faut indiquer `void` (vide) comme type du résultat.
- Une fonction ne peut pas fournir comme résultat des tableaux, des chaînes de caractères ou des fonctions. Il est cependant possible de renvoyer un pointeur sur le premier élément d'un tableau ou d'une chaîne de caractères.

<identificateur > : indique le nom de la fonction qui doit être une lettre ou groupe de lettres et des chiffres commençant obligatoirement par une lettre et ne contenant ni espace ni opérant ni des caractères spéciaux.

Le nom d'une fonction apparaît à trois endroits dans un programme:

- 1) lors de la déclaration
- 2) lors de la définition
- 3) lors de l'appel

Exemple 2

Dans le programme suivant, la fonction `main` utilise les deux fonctions:

- `ENTREE` qui lit un nombre entier au clavier et le fournit comme résultat. La fonction `ENTREE` n'a pas de paramètres.

- `MAX` qui renvoie comme résultat le maximum de deux entiers fournis comme paramètres.

```
#include <stdio.h>
int ENTREE(void);
int MAX(int N1, int N2);
void main()
{
    /* Déclaration des variables */
    int A, B;
    /* Traitement avec appel des fonctions */
    A = ENTREE();
    B = ENTREE();
    printf("Le maximum est %d \n", MAX(A,B));
}
/* Définition de la fonction ENTREE */
int ENTREE(void)
{
    int NOMBRE;
    printf("Entrez un nombre entier : ");
    scanf("%d", &NOMBRE);
    return NOMBRE;
}
```

```

}
/* Définition de la fonction MAX */
int MAX(int N1, int N2)
{
if (N1>N2)
return N1;
else
return N2;
}

```

<arguments> : indique types et noms des paramètres en entrée. Types et nombre des paramètres doivent être compatibles avec celles envoyées dans l'appel de la fonction.

Remarques :

- Nous pouvons utiliser les mêmes noms dans l'entête et l'appel.
- Si une fonction n'a pas de paramètres, on peut déclarer la liste des paramètres comme (void) ou simplement comme ().

<déclarations locales> : déclaration des variables locales à une fonction. En effet, une variable locale ne peut être accessible et visible qu'à l'intérieur de la fonction. En se référant à l'exemple1, les variables `fact` et `i` représentent des variables locales dans la fonction `factorielle`.

<instructions> : représentent les traitements à effectuer dans la fonction. Parmi les instructions, nous pouvons réaliser des appels à d'autres fonctions.

Remarque :

L'imbrication de fonction n'est pas autorisée dans le langage C ; c.à.d. une fonction ne peut pas être définie à l'intérieur d'une autre fonction. Par contre, une fonction peut appeler une autre fonction. Cette dernière doit être déclarée avant celle qui l'appelle.

<return> : Le mot clé `return` suivi d'une expression est le résultat d'une fonction.

Exemple :

```

Float Calcul ( float x, int a, int b)
{ return ( x*x + a*x +b);
}

```

Il est possible d'utiliser l'instruction `return` n'importe où et plusieurs fois dans une fonction.

Exemple :

```

Double somab (double a, double b)
{double S;
S=a+b ;
If(S>a) return S;
Else return (-S);
}

```

L'instruction return a les effets suivants :

- Evaluation de l'expression
- Renvoi du résultat

3. Les variables globales

Les variables déclarées au début du fichier, à l'extérieur de toutes les fonctions sont disponibles à toutes les fonctions du programme. Ce sont alors des variables globales.

En général, les variables globales sont déclarées immédiatement derrière les instructions `#include` au début du programme. Il est à noter que les variables déclarées au début de la fonction principale `main` ne sont pas des variables globales, mais elles sont locales à `main`.

En se référant à l'exemple 1, la variable `réponse` est considérée comme une variable locale.

4. Les paramètres d'une fonction

4.1. Les étapes d'appel d'une fonction

- a- La liste des paramètres effectifs est évaluée, leurs valeurs sont affectées aux paramètres formels de la fonction. Tant qu'on n'a pas atteint la fin de liste des paramètres formels, on recopie la valeur d'un paramètre effectif dans le paramètre formel de même rang.
- b- La fonction invoquée reçoit le contrôle du programme et commence à exécuter ses instructions.
- c- La fonction se termine lorsque l'une des fonctions `return` est exécutée. Si la fonction est de type "void", la fonction se termine lorsqu'on exécute toutes ses instructions.

4.2. Passage de paramètres

Lors de l'appel d'une fonction, la liste des paramètres effectifs est mise en correspondance avec celle des paramètres formels (les paramètres formels et effectifs doivent s'accorder en type, ordre et nombre).

Il y'a deux modes de passage de paramètres :

- Passage par valeur
- Passage par adresse (référence)

Passage par valeur

Dans un passage de paramètres par valeur lors de l'appel de la fonction, la valeur de chaque paramètre est recopiée dans un nouvel espace mémoire réservé pour ce paramètre. Une fonction peut donc modifier localement le contenu de ce paramètre et les valeurs utilisées lors de l'appel ne seront pas modifiées.

Exemple :

```
#include<stdio.h>
void incremente (int x, int y)
{
    x++ ;
    y++ ;
    printf("%d \t %d ", x, y) ;
}
void main()
{
    int a=5, b=7 ;
    printf ("\n Avant appel : %d \t %d ", a,b) ;
    incremente (a, b) ;
    printf ("\n Après appel : %d \t %d ", a,b) ;
}
```

Lors de l'exécution :

Avant appel : 5 7

Après appel : 5 7

Main()

a
5

b
7

Avant appel

Incremente (a, b) copie de a

5
...

6

copie de b

7
...

8

Main()

5

7

Après appel

Passage par adresse

Le passage de paramètre par valeur ne permet pas de modifier une variable de la fonction appelante. Il semble donc que la seule interaction que peut avoir une fonction avec l'extérieur est le renvoi de son résultat.

Grâce aux pointeurs, il est possible d'agir à distance, il suffit pour cela d'envoyer non pas la valeur d'une variable mais son adresse en mémoire, c.à.d. un pointeur sur cette variable.

La fonction appelée peut alors modifier le contenu de cet emplacement mémoire donc le contenu de la variable elle-même.

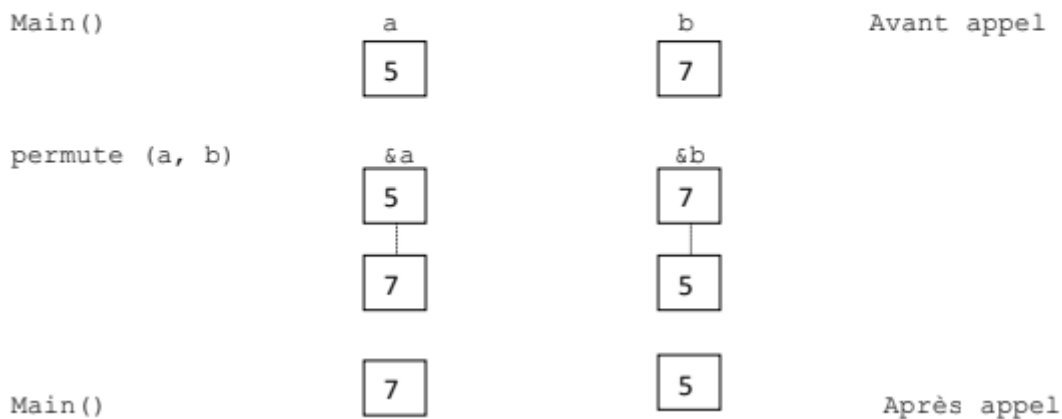
Exemple :

```
#include<stdio.h>
void permute (int *x, int *y)
{
    int Temp ;
    Temp = *x ;
    *x = *y ;
    *y = Temp ;
}
void main()
{
    int a=5, b=7 ;
    printf ("\n Avant appel : %d \t %d ", a,b) ;
    permute (&a, &b) ;
    printf ("\n Après appel : %d \t %d ", a,b) ;
}
```

Lors de l'exécution :

Avant appel : 5 7

Après appel : 7 5



Remarque

/*déclaration d'un pointeur */
Type *nom_variable ;

Exemple

```
int *N ; /*déclarer un pointeur sur une variable de type entier*/
```

Et dans ce cas, N correspond à une adresse. Pour accéder à sa valeur, nous utilisons la notation suivante : *N

N=adresse

*N=valeur de N

Exercice 1

Ecrire un programme C qui permet :

- remplir deux tableaux tab1 et tab2 par des entiers, sachant que la taille maximale de chaque'un est 100.
- afficher tab1 et tab2
- fusionner par alternance les deux tableaux en un seul tableau TF.
- afficher le tableau résultat TF

Exemple

Tab1	14	15	16	17	18		
Tab2	20	30					
TF	14	20	15	30	16	17	18

```
#include <stdio.h>
int TF[200];
void controle (int *n);
void saisi (int n, int T[]);
void affiche (int tab[], int n);
void concat (int tab1[], int tab2[], int n1, int n2);
void main () {
    int n1, n2;
    int tab1[100], tab2[100];

    printf ("donner taillle du premier tableau");
    controle (&n1);
    printf ("saisir les éléments du premier tableau \n");
    saisi (n1, tab1);
    affiche (tab1,n1);
    printf ("donner taillle du deuxième tableau");
    controle (&n2);
    printf ("saisir les éléments du premier tableau \n");
    saisi (n2, tab2);
    affiche (tab2,n2);
    concat(tab1, tab2, n1, n2);
}
void affiche (int tab[], int n)
{
    for(int i=0;i<n;i++)
    printf(" %d |",tab[i]);
}
void saisi (int n, int T[])
{
    for(int i=0;i<n;i++) {
```

```

printf("donner élément d'indice %d  ", i);
scanf("%d",&T[i]);
}
}
void controle (int *n)
{
do
{

    scanf("%d", n);
}while( (*n)<0) || (*n)>100));
}
void concat (int tab1[], int tab2[], int n1, int n2)
{
    int nf=0;
    int i=0;
    int j=0;
    while ((i<n1)&&(j<n2)){
    TF[nf]=tab1[i];
    nf++;
    TF[nf]=tab2[j];
    nf++;
    i++;
    j++;
    }
    for(int k=i;k<n1;k++){// for(;i<n1;i++)
    TF[nf]=tab1[k];
    nf++;
    }
    for(int k=j;k<n2;k++){// for(;j<n2;j++)
    TF[nf]=tab2[k]; //TF[nf]=T2[j]
    nf++;
    }
    //return TF;
    printf("\n Contenu du Tableau TF :");
    affiche(TF, nf);
}

```

Exercice 2

Ecrire un programme qui permet :

- de remplir un tableau T1 par des entiers triés, sachant que la taille réelle du tableau est $1 \leq N \leq 20$.
- afficher les éléments du tableau T1
- vérifier l'existence d'un entier premier dans le tableau

```

#include <stdio.h>
#include <stdbool.h>
void controle (int *n);
void saisi (int T[], int n );
void affiche (int tab[], int n);
bool premier(int x);
bool existe(int tab[], int n);
void main () {
    int n;
    int T1[20];

    printf ("donner taillle du tableau");
    controle (&n);
    saisi(T1,n );
    affiche (T1,n);
    bool R= existe(T1, n);
    if (R== true)
printf ("il existe un entier premier dans le tableau");
else
printf("il n'existe pas un entier premier");
}
void affiche (int tab[], int n)
{
    for(int i=0;i<n;i++)
printf(" %d |",tab[i]);
}
void saisi (int T[], int n)
{ printf("donner élément d'indice 0 ");
scanf("%d",&T[0]);
for(int i=1;i<n;i++) {
    do
    {
printf("donner élément d'indice %d ", i);
scanf("%d",&T[i]);
}while (T[i]<T[i-1]);
}
}
void controle (int *n)
{
do
{
    scanf("%d", n);
}while( (*n)<0) || (*n)>20));
}
bool existe(int tab[], int n)
{
    int i=0;
    bool trouve=false;
    while((i<n)&& (trouve== false))
    {
        if (premier(tab[i])==true)

```

```

        {
            trouve=true;
        }
        i++;
    }
    return trouve;
}
bool premier(int X)
{int i=2;
    bool verif= true;
    while ((i<=X/2)&& (verif==true))
    {
        if (X%i==0)
        {
            verif=false;
        }
        i++;
    }
    return verif;
}

```